My Offer Agent

AI 기반 취업 파이프라인

기술 가이드북 & 프로젝트 문서

김찬중 (Chanjoong Kim)

chanjoongx@gmail.com

버전 1.0.0 | 2026년 3월

프로젝트 정보	
데모	myofferagent.com
기술 스택	Next.js 16, OpenAI Agents SDK, Tailwind CSS v4
배포	Cloudflare Pages
라이선스	MIT

목차

1. 프로젝트 개요
2. 핵심 기능
3. 에이전트 아키텍처
4. 도구 정의
5. API 설계
6. 기술 아키텍처
7. ATS 점수 시스템
8. 배포
9. 다국어 지원

1. 프로젝트 개요

My Offer Agent란?

My Offer Agent는 취업 준비자가 이력서 작성부터 채용공고 검색, 맞춤형 지원서류 준비까지 전체 지원 프로세스를 하나의 채팅 인터페이스에서 관리할 수 있는 대화형 AI 플랫폼입니다. 6개의 전문 AI 에이전트가 협력하여 작동합니다.

주요 기능:

- 자동 핸드오프를 지원하는 6개 전문 AI 에이전트
- 웹 검색 API를 통한 실시간 채용공고 검색
- ATS (지원자 추적 시스템) 100점 만점 이력서 분석
- 특정 채용공고에 맞춤화된 커버레터 및 이력서 생성
- 클라이언트 측 PDF 파싱 — 제3자 서비스로 데이터 전송 없음

해결하는 문제

취업 준비자들은 이력서 빌더, 채용 사이트, ATS 체커, 커버레터 생성기 등 여러 도구를 따로따로 사용해야 합니다. 각 도구마다 다른 인터페이스, 데이터 포맷, 학습 곡선이 있어 시간이 낭비되고 지원 전략에 공백이 생깁니다.

My Offer Agent는 이 모든 워크플로를 하나의 대화형 인터페이스로 통합합니다. 사용자는 필요한 것을 설명하기만 하면, 시스템이 자동으로 적절한 전문 에이전트로 라우팅합니다.

대상 사용자

- 여러 포지션에 지원서를 준비하는 취업 준비자
- 처음 취업 시장에 진입하는 학생
- 새로운 분야를 위해 이력서 최적화가 필요한 이직희망자
- 영문 이력서 작성에 도움이 필요한 국제 지원자

2. 핵심 기능

기능	설명
스마트 라우팅	정규식 사전 라우팅 + LLM 기반 트리아지를 통한 의도 감지 및 에이전트 선택
이력서 빌더	대화형 단계별 이력서 생성 (Markdown 출력)
이력서 분석기	6개 카테고리 100점 만점 ATS 점수 + 개선 피드백
Job Scout	사용자 프로필에 맞는 채용공고 실시간 웹 검색
Match Strategy	이력서 vs. 채용공고 분석 + 키워드 갭 식별
지원서 작성기	채용공고별 맞춤 커버레터 + 최적화된 이력서 변환
PDF 업로드	pdfjs-dist를 통한 클라이언트 측 PDF 파싱 (서버 처리 없음)
다크/라이트 테마	시스템 설정 감지 + 수동 토글
한국어/영어	런타임 언어 전환 지원

주요 차별점

- **통합 인터페이스:** 모든 취업 도구를 하나의 채팅에서 — 앱 전환 필요 없음
- **자동 에이전트 핸드오프:** 사용자는 어떤 에이전트를 사용할지 몰라도 됨; 시스템이 지능적으로 라우팅
- **실시간 채용 데이터:** 오래된 데이터베이스가 아닌 라이브 웹 검색
- **프라이버시 우선 PDF 처리:** 이력서 파싱이 브라우저에서 완전히 처리됨
- **오픈 소스:** MIT 라이선스로 GitHub에 전체 코드 공개

3. 에이전트 아키텍처

6-에이전트 파이프라인

Triage → Builder → Analyzer → Scout → Match → Writer

시스템은 **OpenAI Agents SDK** (@openai/agents)를 사용하여 자동 핸드오프 라우팅을 구현합니다. 각 에이전트는 자체 시스템 프롬프트, 모델, 도구, 핸드오프 대상을 가진 전문 단위입니다.

에이전트	모델	도구	핸드오프	역할
Triage	gpt-4o-mini	없음	→ Builder, Analyzer, Scout	진입점. 의도 파악 후 전문 에이전트로 라우팅.
Resume Builder	gpt-4o-mini	generateResumeMD	→ Analyzer	대화형 이력서 작성.
Resume Analyzer	gpt-4o	parseResume, calculateATS	→ Scout	6개 카테고리 ATS 점수 분석.
Job Scout	gpt-4o	webSearch	없음	실시간 웹 검색으로 채용공고 탐색.
Match Strategy	gpt-4o	없음	→ Writer	이력서 vs. 채용공고 적합성 분석.
Application Writer	gpt-4o-mini	generateResumeMD	없음	맞춤 커버레터 + 최적화 이력서 생성.

스마트 라우팅 로직

라우팅은 두 단계로 진행됩니다:

1단계: 서버 측 정규식 사전 라우팅

LLM 호출 전에 API 라우트에서 키워드 패턴을 검사합니다:

```
// 예시 패턴 (한국어 + 영어 지원)
/채용|공고|검색|찾아|잡|인턴/  →  Job Scout
/이력서.*만들|작성|생성/        →  Resume Builder
/이력서.*분석|ATS|점수/          →  Resume Analyzer
/커버.*레터|자기.*소개서/         →  Application Writer
```

2단계: LLM 기반 트리아지 (Fallback)

정규식으로 매칭되지 않으면, Triage Agent가 GPT-4o-mini를 사용하여 문맥을 이해하고 적절한 전문 에이전트로 전환합니다.

모델 전략

작업 유형	모델	이유
빠른 작업 (트리아지, 빌더, 작성기)	gpt-4o-mini	낮은 지연, 낮은 비용, 충분한 품질
정밀 작업 (ATS 점수, 매칭, 채용 검색)	gpt-4o	분석 작업에 높은 정확도

4. 도구 정의

에이전트는 4개의 도구를 사용하여 외부 데이터와 상호작용하고 전문 작업을 수행합니다:

parseResumeText

항목	세부사항
입력	이력서 텍스트 (string) + 선택적 목표 직무
출력	구조화된 JSON: contactInfo, education, experience, projects, skills
모델	gpt-4o-mini
검증	Zod 스키마 (런타임 타입 검사)
사용 에이전트	Resume Analyzer

calculateATSScore

항목	세부사항
입력	ParsedResume JSON + 목표 직무
출력	ATS 분석: 종합 점수(0-100), 6개 섹션 점수, 강점, 개선사항
모델	gpt-4o (정밀 분석용)
점수 배분	포맷(20) + 키워드(25) + 성과(20) + 구조(15) + 가독성(10) + 문법(10) = 100
사용 에이전트	Resume Analyzer

generateResumeMarkdown

항목	세부사항
입력	ParsedResume JSON
출력	ATS 최적화 Markdown 이력서
모델	gpt-4o-mini (temperature: 0.4)
사용 에이전트	Resume Builder, Application Writer

webSearchTool

항목	세부사항
제공자	OpenAI 내장 웹 검색 도구
입력	Job Scout 에이전트가 구성한 검색 쿼리
출력	실시간 웹 검색 결과 (채용공고)
제약	포괄적 결과를 위해 요청당 최소 2회 검색
사용 에이전트	Job Scout

5. API 설계

엔드포인트

`POST /api/agent`

요청 스키마

```
{
  messages: Array<{
    role: 'user' | 'assistant';
    content: string;
  }>;
  sessionId: string;          // 고유 세션 식별자
  resumeText?: string;        // PDF에서 추출한 텍스트 (선택)
  lastResponseId?: string;    // 대화 체이닝용
  activeAgentName?: string;   // 현재 에이전트 컨텍스트
  language?: 'ko' | 'en';     // 응답 언어
}
```

응답 스키마

```
{
  output: string;              // 에이전트 텍스트 응답
  activeAgent: string;         // 현재 활성 에이전트 이름
  structuredData: {            // 타입된 JSON 페이로드
    atsAnalysis?: ATSAnalysis;
    matchAnalysis?: MatchAnalysis;
    parsedResume?: ParsedResume;
  };
  lastResponseId?: string;     // 다음 요청용
  generatedFiles?: Array<{     // 다운로드 가능한 파일
    type: string;
    content: string;
    fileName: string;
  }>;
  error?: string;
}
```

세션 관리

OpenAI의 `previousResponseId` 메커니즘을 사용하여 대화를 체이닝합니다. 각 응답은 `lastResponseId`를 반환하고, 클라이언트가 다음 요청에 함께 전송하여 서버 측 저장 없이 전체 대화 컨텍스트를 유지합니다.

에러 처리 & 재시도

메커니즘	세부사항
자동 재시도	2회 재시도 + 지수 백오프
재시도 트리거	HTTP 429 (속도 제한), 5xx 오류, 네트워크 장애
타임아웃	에이전트 실행당 120초
검증	모든 도구 입출력에 Zod 스키마 적용

속도 제한

파라미터	값
창	60초
최대 요청	IP당 창당 20회
구현	인메모리 Map (인스턴스당)
본문 크기 제한	500 KB

6. 기술 아키텍처

기술 스택

계층	기술	목적
UI 프레임워크	Next.js 16 (App Router)	React 19 기반 서버 + 클라이언트 렌더링
에이전트 SDK	@openai/agents 0.6.0	멀티 에이전트 오케스트레이션 + 핸드오프
LLM	GPT-4o / GPT-4o-mini	언어 이해 및 생성
스타일링	Tailwind CSS v4	@theme 설정 기반 유틸리티 CSS
PDF 파싱	pdfjs-dist 5.5	클라이언트 측 PDF 텍스트 추출
검증	Zod v4	API 입출력 런타임 타입 검사
컴파일러	React Compiler	자동 메모이제이션 최적화
마크다운	react-markdown + remark-gfm	채팅에서 리치 텍스트 렌더링
아이콘	Lucide React	일관된 아이콘 세트
호스팅	Cloudflare Pages	글로벌 CDN, 엣지 렌더링

프로젝트 구조

```
src/
├── app/
│   ├── page.tsx              # 랜딩 페이지
│   ├── layout.tsx            # 루트 레이아웃 (Theme, i18n, Toast)
│   ├── agent/page.tsx        # 채팅 인터페이스
│   └── api/agent/route.ts    # 에이전트 API 엔드포인트
│
├── lib/
│   ├── agents/
│   │   ├── definitions.ts    # 6개 에이전트 설정
│   │   ├── tools.ts          # 3개 에이전트 도구
│   │   ├── model-config.ts   # GPT 모델 선택
│   │   └── constants.ts      # 에이전트 이름 상수
│   ├── types.ts              # 공유 TypeScript 타입
│   ├── i18n.ts               # 번역 사전
│   └── theme-context.tsx     # 다크/라이트 테마
│
└── components/
    ├── chat/
    │   ├── ChatInterface.tsx  # 메인 채팅 로직
    │   ├── MessageBubble.tsx  # 메시지 렌더링
    │   └── AgentStatusPanel.tsx  # 파이프라인 사이드바
    ├── resume/
    │   ├── ResumeUploader.tsx # PDF 업로드
    │   └── ATSScoreCard.tsx   # 점수 시각화
    └── jobs/
        └── JobCard.tsx         # 채용공고 카드
```

주요 구현 세부사항

- **PDF 처리:** pdfjs-dist를 사용하여 완전히 클라이언트 측에서 처리. 이력서 데이터가 어떤 제3자 서비스로도 전송되지 않음.
- **React Compiler:** 자동 메모이제이션 활성화 — 수동 useMemo/useCallback 불필요.
- **상태 비저장 백엔드:** 이력서 텍스트는 요청 본문으로 전달. 데이터베이스나 파일 저장 없음.
- **대화 체이닝:** OpenAI의 `previousResponseId`로 서버 상태 없이 멀티턴 컨텍스트 유지.
- **Zod 검증:** 모든 도구 입출력을 런타임에 검증하여 비정상 데이터 방지.

7. ATS 점수 시스템

Resume Analyzer는 이력서를 100점 만점으로 6개 카테고리에 걸쳐 평가합니다. 실제 ATS 시스템(Workday, Greenhouse, Lever 등)이 지원자를 평가하는 방식을 반영하여 설계되었습니다.

카테고리	배점	평가 항목
포맷 호환성	20	깔끔한 구조, 표준 섹션, 파싱 가능한 레이아웃, 일관된 포맷팅
키워드 최적화	25	업계 표준 용어, 직무별 키워드, 스킬-직무 정렬
성과 품질	20	정량화된 결과, 액션 동사, 측정 가능한 영향력 서술
구조적 완성도	15	필수 섹션 존재: 연락처, 학력, 경력, 스킬
가독성	10	명확한 언어, 적절한 길이, 논리적 흐름, 불릿 포인트 활용
문법 & 맞춤법	10	오류 없는 글쓰기, 전문적 어조, 일관된 시제

출력 형식

각 카테고리에는 다음이 포함됩니다:

- **점수:** 카테고리 최대점 대비 수치
- **피드백:** 구체적인 강점과 개선 제안
- **예시:** 구체적인 추천 (예: "Agile, Scrum, CI/CD 키워드 추가")

종합 점수에는 다음도 포함됩니다:

- **상위 3개 강점:** 이력서가 잘 하고 있는 부분
- **상위 3개 핵심 개선사항:** 가장 영향력 있는 변경 사항

8. 배포

Cloudflare Pages (기본)

애플리케이션은 OpenNext 어댑터를 사용하여 Cloudflare Pages에 배포됩니다. Next.js 출력을 Cloudflare Workers 호환 형식으로 변환합니다.

구성요소	세부사항
어댑터	@opennextjs/cloudflare 1.17.1
워커	.open-next/worker.js
애셋	.open-next/assets/ (Cloudflare Cache)
호환성	Node.js 호환 플래그 활성화

빌드 & 배포 명령어

```
# 개발
npm run dev          # 로컬 개발 서버 (localhost:3000)

# Cloudflare 배포
npm run cf:build     # Cloudflare용 빌드
npm run cf:preview   # 로컬 미리보기
npm run cf:deploy    # 프로덕션 배포

# 환경 변수
npx wrangler secret put OPENAI_API_KEY
```

대안: Vercel

```
npm run build        # 표준 Next.js 빌드
npm run start        # 로컬 프로덕션 서버
# Vercel 대시보드 → Settings → Environment Variables에서 OPENAI_API_KEY 설정
```

환경 변수

변수	필수	설명
OPENAI_API_KEY	예	에이전트 실행 및 웹 검색용 OpenAI API 키

9. 다국어 지원

My Offer Agent는 한국어와 영어를 지원하며 런타임 언어 전환이 가능합니다. 사용자의 선호는 브라우저에 저장됩니다.

코드	언어	커버리지
en	English	전체 (기본값)
ko	한국어	전체

구현 방식

- **방식:** `i18n.ts`에 flat key-value 사전
- **상태:** React Context (`i18n-context.tsx`) + localStorage 영속성
- **범위:** UI 레이블, 툴팁, 에이전트 사이드바, 에러 메시지, 플레이스홀더
- **에이전트 언어:** 언어 파라미터를 API로 전송; 영어 모드에서는 영어 응답을 강제하는 명령을 추가

URL 기반 로케일 라우팅이나 무거운 i18n 라이브러리 없이 — 이중 언어 애플리케이션에 적합한 경량 접근 방식입니다.

이메일: chanjoongx@gmail.com | GitHub: github.com/chanjoongx/myofferagent